

We don't need to worry about the details of this function since we're just using it to check our continued fractions. We can check that the function works by running `get_number([3,5,2])` and seeing that we get 3.181818... as the output, which is another way to write 105/33 (the number we started with).

From Decimals to Continued Fractions

What if, instead of starting with some x/y as an input to our continued fraction algorithm, we start with a decimal number, like 1.4142135623730951? We'll need to make a few adjustments, but we can more or less follow the same process we followed for fractions. Remember that our goal is to find a, b, c , and the rest of the alphabet in the following type of expression:

$$1.4142135623730951 = a + \frac{1}{b + \frac{1}{c + \frac{1}{d + \frac{1}{e + \frac{1}{f + \frac{1}{g + \frac{1}{\dots}}}}}}}$$

Finding a is as simple as it gets—it's just the part of the decimal number to the left of the decimal point. We can define this `first_term` (a in our equation) and the leftover as follows:

```
x = 1.4142135623730951
output = []
first_term = int(x)
leftover = x - int(x)
output.append(first_term)
```

Just like before, we're storing our successive answers in a list called `output`.

After solving for a , we have a leftover, and we need to find a continued fraction representation for it: